

RepFinder

Relatório Técnico Final

Plataforma de Conexão entre Estudantes e Representantes Comerciais

Pontifícia Universidade Católica de Minas Gerais — PUC Minas
Sistemas de Gerência de Redes e Internet — SGA
Junho • 2026

1. Introdução

O RepFinder é uma plataforma mobile composta por três grandes componentes — um backend RESTful, um aplicativo para estudantes (client) e um aplicativo para representantes comerciais (provider) — cujo objetivo é conectar candidatos a vagas de representação a empresas parceiras.

Este relatório documenta as decisões técnicas tomadas ao longo do desenvolvimento, a arquitetura adotada, os padrões de codificação empregados e as correções de bugs relevantes realizadas durante a fase de integração.

2. Visão Geral da Arquitetura

O sistema segue um modelo de monólito modular no backend, comunicação REST com JWT nos clientes e um canal assíncrono via Redis Pub/Sub para entrega de notificações em tempo real.

2.1 Componentes Principais

Componente	Tecnologia	Responsabilidade
API (Backend)	Node.js + Hono + TypeScript	Regras de negócio, persistência, eventos
App Client	Flutter + Riverpod	Interface do estudante: busca de vagas e candidaturas
App Provider	Flutter + Riverpod	Interface do representante: gestão de vagas e candidatos
Banco de Dados	MySQL 8	Persistência relacional de usuários, vagas e candidaturas
Message Broker	Redis Pub/Sub	Canal de eventos assíncronos entre módulos
CDN de Mídia	Cloudinary	Upload e transformação de imagens de perfil (avatars)

2.2 Fluxo de Requisição (HTTP)

Toda requisição ao backend percorre a seguinte sequência:

1. Recepção e roteamento (Hono Router)
2. Validação de autenticação — middleware JWT (jose + HS256)
3. Delegação ao módulo de domínio correspondente
4. Execução das regras de negócio na Service Layer
5. Acesso ao banco via Repository Layer (mysql2/promise — pool de 10 conexões)
6. Retorno da resposta ao cliente

A publicação de eventos é realizada de forma assíncrona, envolvida em try/catch, para que a indisponibilidade pontual do Redis não afete a resposta HTTP principal.

2.3 Fluxo de Notificação em Tempo Real

Após a persistência no banco, um evento tipado (AppEvent) é publicado no canal Redis `repfinder:events` pelo módulo de applications. O serviço de notificações, inscrito nesse canal via `subscriber` dedicado, recebe o payload, persiste uma notificação na tabela `notifications` e entrega o evento ao cliente conectado por SSE (Server-Sent Events), mantendo a conexão HTTP de longa duração aberta por `WritableStream`.

Nota: Redis Pub/Sub não garante persistência de mensagens. Caso o `subscriber` esteja offline durante a publicação, o evento é perdido. Eventos passados podem ser consultados pela tabela `notifications` no banco, que serve como histórico durável.

3. Backend — API REST

3.1 Stack e Dependências

Pacote	Versão	Finalidade
hono + @hono/node-server	^4.12	Framework HTTP minimalista com tipagem nativa
mysql2/promise	^3.22	Driver MySQL com suporte a async/await e pool
redis	^5.12	Cliente Redis para Pub/Sub (publisher + subscriber)
jose	^6.2	JWT: geração (SignJWT) e verificação (jwtVerify) com HS256
typescript	^5.8 (dev)	Tipagem estática, generics e type safety nos eventos
tsx (dev)	^4.7	Execução TypeScript sem build em desenvolvimento

3.2 Estrutura de Módulos

O backend é organizado em módulos verticais, cada um com as camadas: routes, service e repo.

- users — cadastro, login, perfil público e autenticado, troca de senha
- vacancies — CRUD de vagas, restrito a usuários com role representative
- applications — candidatura, listagem, atualização de status com transição de estado
- notifications — consulta de histórico e marcação de leitura

O arquivo `src/lib/lifecycle.ts` implementa um registry de serviços (interface `Service` com `start/stop`), garantindo que Redis publisher, Redis subscriber e o servidor HTTP sejam iniciados e encerrados de forma ordenada, com tratamento de `SIGTERM` e `SIGINT`.

3.3 Modelo Relacional

O banco de dados contém quatro tabelas principais:

Tabela	PK	Pontos de interesse
users	UUID v4	Senha armazenada como pbkdf2-SHA512 (iterations:salt:hash); role ∈ {student, representative}
vacancies	UUID v4	FK provider_id → users.id; exclusão bloqueada se houver applications
applications	UUID v4	Unique constraint (user_id, vacancy_id); status ∈ {pending, accepted, rejected}; transição irreversível após saída de pending
notifications	UUID v4	Leitura indicada por readed_at (timestamp), não por booleano; payload do evento serializado como JSON em data TEXT

3.4 Autenticação e Segurança

- Senha: PBKDF2-SHA512, 100.000 iterações, salt aleatório de 16 bytes (Node.js crypto.pbkdf2Sync). Comparação com timingSafeEqual para prevenir timing attacks.
- JWT: tokens HS256 com expiração de 7 dias, gerados com SignJWT (jose). Verificação via authMiddleware injetado em todas as rotas protegidas.
- Controle de acesso: verificação de ownership (assertIsOwner) em todas as operações sobre vacancies e applications.
- Logout forçado: interceptor Dio nos clientes Flutter detecta 401 e limpa o token, redirecionando para /login.

3.5 Sistema de Eventos (Pub/Sub)

O módulo de eventos é composto por três artefatos:

- events.types.ts — union type AppEvent cobrindo application.created e application.status.updated, cada um com campos tipados explicitamente.
- event.factory.ts — factory genérica createEvent<E> que injeta o campo occurredAt automaticamente, evitando que cada ponto de uso se preocupe com a serialização do timestamp.
- publisher.ts — implementa a interface Service (lifecycle registry) e publica no canal repfinder:events via PUBLISH do Redis.

```
channel: repfinder:events
```

```
events: application.created | application.status.updated
```

4. Aplicativos Flutter

4.1 Stack e Dependências Compartilhadas

Pacote	Versão	Finalidade
flutter_riverpod	^2.5	Gerência de estado reativa com suporte a AsyncValue
riverpod_annotation + generator	^2.3 / ^2.4	Geração de código para providers (@riverpod)
dio	^5.4	HTTP client com interceptors para auth e cache
dio_cache_interceptor	^3.5	Cache em memória (MemCacheStore, TTL 1h/7d por app)
flutter_secure_storage	^10.3	Armazenamento seguro do JWT (Keychain/Keystore)
shared_preferences	^2.2	Fila offline de candidaturas (OfflineQueue)
connectivity_plus	^6.0	Stream de estado de conectividade (keepAlive provider)
flutter_dotenv	^5.1	Variáveis de ambiente (.env) incluídas como asset
json_serializable	^6.7 (dev)	Serialização/desserialização de modelos via geração de código

4.2 Arquitetura dos Apps (Clean-ish Layers)

Ambos os aplicativos adotam a mesma divisão em camadas:

- core/ — infraestrutura transversal: configuração do Dio (AuthInterceptor, DioCacheInterceptor), armazenamento seguro (SecureStorage), gestão de conectividade e fila offline (apenas client).
- datasources/ — acesso remoto ao backend. Cada datasource é um @riverpod e usa a instância do Dio injetada via dioProvider.
- domain/ — controllers: AsyncNotifier (Riverpod) que orquestram chamadas ao datasource, encapsulam estado e disparam eventos de ciclo de vida.
- models/ — data classes geradas com json_serializable (fromJson / toJson / .g.dart).
- screens/ — telas com ConsumerStatefulWidget ou ConsumerWidget.
- widgets/ — componentes reutilizáveis: VacancyCard, CandidateCard, NotificationItem, etc.

4.3 Funcionalidades por App

App Client (Estudante)

- Login e registro com validação local antes de enviar ao backend
- Listagem de vagas abertas com filtros
- Candidatura a vagas com feedback visual e fila offline (SharedPreferences)
- Acompanhamento de candidaturas com chips de status (pending / accepted / rejected)

- Perfil editável com upload de avatar via Cloudinary (upload direto, sem passar pelo backend)
- Central de notificações com abas 'Todas' e 'Não lidas', contagem de badge e marcação de leitura
- Recepção de eventos SSE (status de candidatura atualizado pelo representante)

App Provider (Representante)

- Login e registro com role representative
- Dashboard de vagas criadas com visualização de candidatos por vaga
- Criação e exclusão de vagas (exclusão bloqueada se houver candidaturas)
- Gestão de candidatos: aceite ou recusa com botões travados durante requisição
- Recarga automática da lista após cada decisão via callback onChanged
- Central de notificações com SSE para novos candidatos em tempo real

5. Padrões de Projeto e Decisões Arquiteturais

5.1 Decisão: Monólito Modular com Worker In-Process

O consumidor de eventos Redis não foi extraído como processo separado. O serviço de notificações (NotificationService) é registrado via lifecycle registry e roda no mesmo processo da API HTTP. Esta escolha reduz a complexidade operacional na fase atual do projeto e facilita o deploy, ao custo de escalar o consumidor junto com o servidor HTTP.

***Nota:** Caso o volume de eventos cresça ou seja necessário reprocessamento, a migração para Redis Streams (com consumer groups) ou um broker com persistência (RabbitMQ, Kafka) é o caminho natural, sem necessidade de alterar a interface Service nem os módulos de negócio.*

5.2 Decisão: SSE ao invés de WebSocket

Server-Sent Events foram preferidos a WebSocket pelo menor overhead de handshake e pela aderência ao modelo HTTP/1.1 sem necessidade de protocolo adicional. Como o fluxo de notificações é unidirecional (servidor → cliente), SSE é suficiente e mais simples de manter. A conexão é aberta pela tela de Notificações em initState e encerrada em dispose (via referência capturada antecipadamente).

5.3 Decisão: Upload Direto para Cloudinary

Imagens de avatar são enviadas diretamente do dispositivo ao Cloudinary via upload preset não assinado (unsigned). O backend nunca recebe o binário da imagem — apenas a URL resultante via PATCH /users/me. Isso elimina carga de processamento no servidor e delega transformações (c_fill, w_400, h_400, f_webp, q_auto) à CDN.

5.4 Padrão: Event Factory Genérico

A função createEvent<E> usa generics discriminados pelo campo event para garantir que o payload de cada tipo de evento seja correto em tempo de compilação. O eventFactory é um objeto simples com métodos nomeados (applicationCreated, applicationStatusUpdated), tornando a criação de eventos legível e evitando construção manual do campo occurredAt em cada ponto de uso.

5.5 Padrão: Service Registry com Lifecycle

O módulo lifecycle.ts implementa um registry simples de serviços (interface Service { name, start, stop? }). Serviços se registram via register() e são iniciados/encerrados em ordem (direto para start, inverso para stop) via startAll/stopAll. Isso garante que o Redis publisher seja encerrado apenas após o subscriber, evitando mensagens perdidas no shutdown.

6. Bugs Identificados e Corrigidos

6.1 Bug: Status 409 em Cascata ao Aprovar/Recusar Candidaturas

Sintoma

Ao tocar nos botões "Aceitar" ou "Recusar" no app Provider, múltiplas requisições PATCH /applications/:id/status eram disparadas em sequência rápida, todas retornando HTTP 409. O log exibia alternância entre status accepted e rejected a cada ~1 segundo.

Causa Raiz

A tela ManageCandidatesScreen criava a Future de listagem de candidaturas diretamente dentro do método build(), o que significa que um novo Future era criado a cada rebuild sem disparar nenhum carregamento real. Adicionalmente, o widget CandidateCard era um ConsumerWidget stateless cujos botões chamavam updateStatus() sem qualquer mecanismo de debounce ou feedback visual de carregamento. Sem feedback, o usuário clicava repetidamente. Como a primeira requisição bem-sucedida já alterava o status de pending para accepted ou rejected, todas as requisições subsequentes violavam a regra de negócio do backend (status imutável após saída de pending) e retornavam 409.

Correção

- ManageCandidatesScreen convertida para ConsumerStatefulWidget. A Future é armazenada em estado (_applicationsFuture) e o método _load() é chamado explicitamente no initState e após cada decisão via callback onChanged.
- CandidateCard convertido para ConsumerStatefulWidget com flag booleana _isSubmitting. Os botões são desabilitados (onPressed: null) durante a requisição em andamento. Um indicador de progresso substitui o rótulo do botão.
- O callback onChanged é sempre invocado no bloco finally, garantindo que a lista recarregue mesmo em caso de erro, mantendo a UI sincronizada com o estado real do servidor.
- O applicationControllerProvider eliminado o ref.invalidateSelf() ineficaz, já que nenhuma tela observava aquele provider com watch.

Antes: `ref.invalidateSelf()` – nenhum efeito (provider não era observado)

Depois: `widget.onChanged()` – `_load()` na tela recarrega do servidor

6.2 Bug: StateError ao Desmontar NotificationsScreen

Sintoma

Ao sair da tela de notificações (especialmente em situações de desmontagem em lote da árvore de widgets, como logout ou navegação com pop em cadeia), o Flutter lançava: StateError: Bad state: Cannot use 'ref' after the widget was disposed.

Causa Raiz

O método dispose() de _NotificationsScreenState continha a chamada `ref.read(notificationControllerProvider.notifier).stopListening()`. O Riverpod proíbe o uso de ref após o elemento ter sido marcado como desmontado pelo Flutter. Em situações onde a desmontagem ocorre em cascata (_InactiveElements._unmount em lote), o Flutter já marca o elemento como disposed antes de executar dispose(), tornando o acesso a ref inválido naquele ponto.

Correção

A referência ao notifier foi capturada antecipadamente como campo da classe, inicializado por um inicializador late (late final `_notificationsNotifier = ref.read(...)`) que é executado na primeira vez que o campo é acessado — dentro do contexto de `initState`, quando `ref` ainda é válido. O `dispose()` passa a usar `_notificationsNotifier.stopListening()` sem tocar em `ref`.

***Nota:** Isso se aplica a qualquer `ConsumerStatefulWidget` que precise chamar métodos de um notifier no `dispose()`. A regra geral é: nunca acessar `ref` dentro de `dispose()`; use campos capturados em `initState`.*

A correção foi aplicada em ambos os apps (client e provider), pois o mesmo padrão estava replicado.

7. Análise Crítica e Pontos de Atenção

7.1 Pontos Fortes

- Separação clara de responsabilidades em todas as camadas (routes → service → repo no backend; datasource → controller → screen no frontend).
- Sistema de eventos desacoplado: o módulo de applications não conhece o módulo de notifications — comunica-se apenas via Redis.
- Transições de estado irreversíveis na camada de serviço garantem integridade dos dados mesmo sob concorrência.
- Armazenamento seguro de token com flutter_secure_storage (Keychain/Keystore por plataforma).
- Fila offline para candidaturas (OfflineQueue) com flush automático ao reconectar.
- Upload de mídia sem passar pelo servidor, reduzindo latência e banda.

7.2 Riscos e Melhorias Recomendadas

Risco / Limitação	Impacto	Recomendação
Redis Pub/Sub sem persistência	Notificação perdida se subscriber offline	Migrar para Redis Streams com consumer groups
Conexões SSE por usuário no Map<userId, Writer>	Apenas uma sessão ativa por usuário	Mapear userId → List<Writer> para multi-device
URL da API hardcoded (localhost) no client app	Não funciona fora do emulador	Usar flutter_dotenv também no client (já adotado no provider)
Ausência de testes automatizados	Regressões difíceis de detectar	Adicionar testes unitários nos services e widget tests nos controllers
JWT sem refresh token	Usuário deslogado após 7 dias sem alternativa	Implementar refresh token com rotação
Consumidor in-process	Escalabilidade acoplada ao servidor HTTP	Extrair NotificationService como worker autônomo

8. Referências Técnicas

8.1 Backend

- Hono — documentação oficial: <https://hono.dev>
- node-redis v5 — documentação oficial: <https://github.com/redis/node-redis>
- JOSE (JavaScript Object Signing and Encryption): <https://github.com/panva/jose>
- mysql2 — pool de conexões e prepared statements: <https://github.com/sidorares/node-mysql2>
- Redis Pub/Sub vs. Streams — Redis docs: <https://redis.io/docs/manual/pubsub/>
- Server-Sent Events — MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- PBKDF2 — NIST SP 800-132 (recomendação de 100.000+ iterações)

8.2 Flutter / Dart

- Flutter Riverpod — documentação oficial: <https://riverpod.dev>
- riverpod_annotation / riverpod_generator — geração de código: https://pub.dev/packages/riverpod_generator
- Dio — HTTP client para Dart: <https://pub.dev/packages/dio>
- dio_cache_interceptor: https://pub.dev/packages/dio_cache_interceptor
- flutter_secure_storage — armazenamento seguro: https://pub.dev/packages/flutter_secure_storage
- connectivity_plus: https://pub.dev/packages/connectivity_plus
- ConsumerStatefulWidget e ciclo de vida — Riverpod docs: <https://riverpod.dev/docs/concepts/lifecycle>
- Bad state: Cannot use ref after widget disposed — Issue rastreado: <https://github.com/rrousselGit/riverpod/issues/1303>

8.3 Infraestrutura e Padrões

- Cloudinary Upload API — unsigned presets: https://cloudinary.com/documentation/upload_images#unsigned_upload
- DBML — Database Markup Language: <https://dbml.dbdiagram.io>
- Clean Architecture (adaptada) — Robert C. Martin, 2017
- Event-Driven Architecture — Martin Fowler, 2017: <https://martinfowler.com/articles/201701-event-driven.html>
- The Modular Monolith — Kamil Grzybek: <https://www.kamilgrzybek.com/blog/posts/modular-monolith-primer>